Lab 6 Report

1 Introduction

For our Lab 6 project, we implemented garbage collection on our L4 compiler. In accordance with the recommendation from the writeup, we implemented a *copying collector*.

1.1 Implementation

The general strategy of our copying collector is to split the heap into two spaces: the *from-space* and the *to-space*. Every time `alloc` is called, if there is not enough memory left to allocate, the collector sweeps through the from-space, starting from a set of *root pointers*, and copies anything visited to the to-space. Anything not copied into the to-space can be safely deleted. The implementation of the copying collector can be split into two sections: compile-time and run-time.

During compile-time (see Section 2), the compiler stores in the stack frame the type of each temp on the stack, and stores in headers the type of each heap-allocated object. This allows the copying collector to inspect the stack for the set of root pointers and to traverse the heap for the set of live objects. The compiler also replaces calls to `calloc` with calls to `gc_alloc`, which implements garbage collection.

During run-time (see Section 3), the copying collector initially allocates space on the heap for the from- and to-spaces. Every time `gc_alloc`, if the amount of space requested is not available, the collector sweeps through the stack using the stack traversal algorithm to find a set of root pointers. Then the collector uses Cheney's algorithm to perform a breadth-first search through the heap, copying anything visited into the to-space. If the collector still cannot allocate enough space, it will fail; otherwise, it will allocate space for the object and return a pointer to the start of the object.

1.2 Results

We found that the garbage collector successfully collects any objects which no longer have pointers to them. This ensures memory safety, but also means that objects whose pointers are never used again in a function are not collected until the function returns. However, this is enough for the garbage collector to make executing programs that allocate a lot of space (see test case `stress01.l4`) possible, whereas with L4 they would run out of memory. We also found that the garbage collector does not result in memory leaks since we are able to check the type of everything stored on the stack.

In terms of performance, adding garbage collection slowed down compile time when we have large nested structs due to adding header information necessary for collection. Similarly, adding this information slowed down the execution of the program, and increased the size of the stack frames, limiting the amount of recursion possible before a stack overflow.

Adding extra information also means that we no longer comply with C standards for array and struct layouts, which causes our compiler to fail tests that link against and call C functions on C0 constructs.

2 Compilation

Being able to follow pointers is necessary for a copying collector to do its job. Because our garbage collector runs during runtime, we stored information we had at runtime about the types of the objects stored on the stack and on the heap. The compiler now generates code to store this information at the same time as it allocates an object or assigns a value to a temp.

2.1 Heap Objects

Objects stored on the heap are given an 8-byte header that indicates their type, as well as any extra information necessary for traversal during the collection phase. By inserting a tag into the header, we can determine the type of an object at runtime. We categorize heap-allocated objects using four tags: `data`

(Figure 1), `pointer` (Figure 2), `array` (Figure 3), and `struct` (Figure 4). Changes to the layout of heap-allocated objects are documented in their figure captions.

A heap-allocated object's header is inserted upon allocation. Objects are handled in a similar way to the way arrays in L4 were handled; `gc_alloc` is called on enough space for both the object and its header, and the returned pointer is moved up 8 bytes to point to the object rather than the header. Because types are known at compile time, the compiler now generates code to store header information at the same time as it generates code to call `gc_alloc`. For data and pointers, this only results in one extra write, and for arrays, this results in three extra writes. For structs, however, the headers and sizes of each field have to be inserted at compile time, since they are all initialized to zero when `calloc` is called. This results in very long allocation time for severely nested structs.

See the appendix for example code and corresponding heap objects.

2.2 Stack Structure

The layout of a c0 function's stack frame was updated to encode sufficient information for marking and tracking pointers over the stack frames of a program. At a high level, the following changes were made:

1. The stack base pointer (%rbp) was removed as an assignable program register and instead used to store the active function's base frame stack address
2. Type annotations for stack variables were added as a stack frame section (typing annotations) to allow identification of pointer variables on the stack
3. Pointer variables were restricted to live on the stack
4. The count of stack variables was added as a stack frame section (stack data count) to aid manipulation of a function's stack frame from an external c function

2.2.1 Base Pointer

Immediately after being called, a function saves the caller function's base pointer by pushing it to the stack, similar to the "callee-save" register procedure. The base pointer is restored prior to returning from the callee.

The implementation of this change was vital to facilitating the "stack-traversal" algorithm recommended for the copying collector. Thus, following the general garbage collection theme of feasibility and safety over efficiency, the extra register was sacrificed to maintain the necessary information for stack frame traversal of the program.

2.2.2 Count of Stack Variables

In order to traverse the stack data of a function during the runtime of garbage collection, it was necessary to save the count of stack variables. This was pushed at the function's base for easy access during function stack traversal.

2.2.3 Type Annotations

Much like heap object headers, local variables stored in the function stack frames also required type annotations to distinguish stack slots storing a memory address and stack variables storing immediate data. However, stack frames must adhere to a stricter data layout to uphold calling conventions, and have more limited space availability. Therefore, instead of concatenating every 8-byte stack slot with a header containing the slot's corresponding typing information, after consultation with course instructors, a design decision to maintain typing information using a separate mapped region was made.

The mapped region was implemented as a sequence of integer stack slots positioned under the section allocated for stack data. Each integer represented a binary sequence where every two-bits corresponded to a particular program variable stored on the stack, indexed by the position of that variable's stack slot. To maintain the correctness of the typing information as new variables were introduced into stack slots after

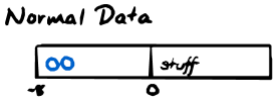

Figure 1: Layout of `data`-tagged objects. Blue indicates binary numbers. Any heap-allocated object that has type `int` or `bool` has the tag `data`. These objects are always 8-byte aligned, so with the 8-byte header and 8-byte data they take 16 bytes of memory.

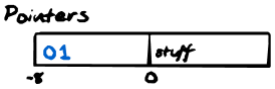

Figure 2: Layout of `pointer`-tagged objects. Blue indicates binary numbers. Any heap-allocated object that has type τ`*` has the `pointer` tag. Additionally, objects with type τ`[]` have a `pointer` tag, since they are stored as pointers to the start of the array. These objects also take 16 bytes of memory.

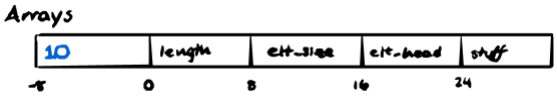

Figure 3: Layout of `array`-tagged objects. Blue indicates binary numbers. Array objects are the only ones with the `array` tag. The layout of arrays have additionally been changed to make collection more convenient. The pointer to an array, rather than pointing to the first element of the array, points to the length of the array.

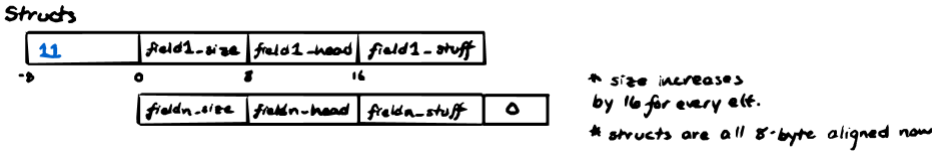

Figure 4: Layout of `struct`-tagged objects. Blue indicates binary numbers. Each field is prefixed with its size (where the size is inclusive of itself, the header, and the field, so a field of size 8 would have a size of 24). Structs are also now all 8-byte aligned. Structs always end in a null terminator.

the liveness of previous variables ended, the bits were updated during the runtime of the program using bit masks and typing information of operands available at compiler time. To do so, every assembly move instruction into one of the stack slots in the function's data section was prepended by an update to the corresponding slot bits based on the type of the new variable.

This additional bit masking operation with every move on to the stack added a large amount of overhead to the runtime of the program, causing an increase in test-case timeouts as discussed in 5.3.2.

2.2.4 Pointers Restricted to Live on Stack

The L4 c0 compiler was capable of storing pointers in registers. Originally, the L6 compiler intended to maintain this feature. To facilitate this, the types of register arguments was stored in each stack frame's typing annotations section. However, during implementation of the garbage collection, maintaining the capability of remapping register pointers became much more challenging. The primary difficulty lay in tracking and distinguishing the typing of data actively changing *between* function calls, including on call of external functions and the garbage collection function written in c and compiled down by gcc. This led to unpredictable register use behavior during runtime on function calls and thus difficulty in tracking the types of register values from within the garbage collector. To maneuver around this, pointer values were restricted from being allocated into a register in the register allocator, and instead forced onto the stack, where typing information could be safely maintained between function calls.

3 Runtime System

To implement garbage collection, we made significant changes to `run411.c` and the runtime environment. Garbage collection and allocation can be found from lines 34 through 506, and stack traversal can be found in `run411.c`.

3.1 Allocation

Prior to calling `c0_main`, `run411.c` now calls `gc_init`, which initializes the heap by allocating memory for both the from- and to-spaces. "Free" memory is identified by the `next_free` pointer, which points to the free memory in the from-space. Every time "allocation"' happens, the `next_free` pointer simply moves up in the from-space. This means that our programs always use a constant amount of memory until they finish.

Rather than calling `calloc`, the compiler now generates calls to `gc_alloc`, which implements the garbage collection. `gc_alloc` does the following to allocate memory with garbage collection:

1. Check if there is enough space in the from-space to allocate. If not, call the garbage collecting function (`collect`).
2. Check again if there is enough space to allocate. If not, there will never be enough space, so abort the program.
3. Allocate space and update the `next_free` pointer.

3.2 Stack Traversal Algorithm

The stack was traversed as follows:

1. The top stack frame's rsp and rbp pointers were passed to the garbage collector
2. The stack variable slot count for the current stack frame was read
3. Using this slot count, the typing annotations for each slot index were extracted from the stack using bit masking

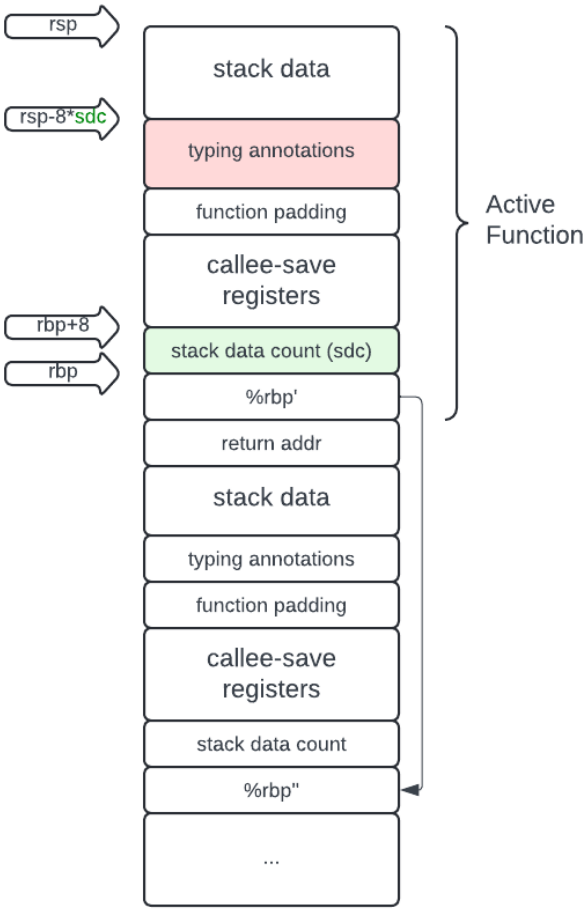

Figure 5: Updated stack function layout for compatibility with garbage collection

4. Pointers, arrays, and structs (the c0 heap-allocated objects) marked 01, 10, and 11 were indexed and added to the root pointers used in Cheney's.
5. If the current stack frame was c0_main, the stack traversal was terminated, otherwise the rsp and rbp pointers for the following stack frame were recovered from the stack and the traversal returned to step 2.

3.3 Cheney's Algorithm

The copying collection algorithm we used (given a set of root pointers from the stack traversal algorithm) was Cheney's algorithm. We referenced the Wikipedia page and the Garbage Collection Handbook to implement this. Cheney's algorithm uses a breadth-first search to traverse pointers in the heap, starting from the root set:

1. Swap the to- and from-spaces.
2. Obtain a set of root pointers from the stack traversal algorithm.
3. For every root pointer, copy the object it points to from the old from-space to the new from-space and update the root pointer to point to the new location. If it has already been copied, the object will be a *forwarding address*, which is represented as a pointer to the new location, so just update the root pointer without copying again.
4. Iterate over the new from-space by keeping track of a `scan_ptr`, which starts at the beginning of the from-space. Keep iterating until `scan_ptr` catches up with `next_free`, meaning everything in the from-space has been iterated over.

   For each object `scan_ptr` points to:

   (a) Check the object's header to see if it contains any internal pointers. If not, skip it.
   (b) For every pointer inside the object, try to copy it into the new from-space and update the pointer. If it has already been copied (and is therefore a forwarding address), just update the pointer.
5. Clear the to-space by setting every bit to zero. Anything not copied into the from-space has now been collected.

4 Testing

Our test suite includes two subfolders. `functionality` new test cases written specifically to test the functionality of the garbage collector, and `regression` contains test cases from L1 through L4. To run the test cases, copy `run411.c` into `dist` and run:

```
../gradecompiler --limit-run=120 lab6-tests/functionality/
```

4.1 Test Cases

In order to test the functionality of the garbage collector, we identified several goals for our collector:

1. **Correctly execute code from L1 through L4.** In order to test correct execution, we added every basic test case from L1 through L4 except `helloworld.l4` (see Section 4.2.3), and several large test cases from L1 through L4.
   - For L1, mostly stress tests were chosen, since not much else could be tested.
   - For L2, large tests and tests with branching were chosen to make sure that the compiler still generates correct code.
   - For L3 and L4, we chose tests which mimicked actual programs, rather than deliberate stress tests, since our increased overhead due to our garbage collector causes us to fail many of these. We also chose tests that had multiple arguments, and tests that resulted in memory errors to make sure that L3 and L4 are correct.

- We included our own test case, `frog_village`, because I spent a long time on it and I want someone to see it.

$$>: 3$$

See section 4.2 for the results of full regression testing.

2. **Collect objects when no pointers exist to them.** In particular, the collector should be able to collect anything allocated inside a function that has returned, unless a pointer was returned out of the function.

3. **Avoid collecting objects that might be used again.** To make sure that our garbage collector never collects objects prematurely, it only collects objects after no pointer to them exists in the stack. This means that our garbage collector will not collect pointers in a function until that function returns, even if the pointers went out of scope.

4. **Perform correctly under stress.** Even if forced to do many collections, or one collection every allocation, the garbage collector should be able to return the correct value. These stress tests force large numbers of collections.

5. **Avoid memory leaks**. Because we implemented a full copying collector with type annotations (at the expense of performance, compile time, and space), we should not have memory leaks in our programs.

4.1.1 Cheney Scan

The file `gc_mem.c` contains seven simple test cases for testing Cheney's algorithm (the `collect` function). These test cases can be run by compiling and running `gc_mem.c`, since the file only runs `collect`.

4.1.2 Heap Correctness

Several tests in the test suite are present to ensure that heap objects have correct headers and information after allocation. These tests all have the naming pattern `heapXX.l4`, and require use of `gdb` to manually verify that objects look as expected.

4.2 Regression Testing

Running L1 through L4 on our compiler with 10MB of available memory, we have the following results:

Text Suite	Results
`l1-basic`	16/16
`l1-large`	3473/3473
`l2-basic`	29/29
`l2-large`	5316/5318
`l3-basic`	36/36
`l3-large`	4668/4821
`l4-basic`	43/44
`l4-large`	5680/5724

The reasons for failures are discussed below.

4.2.1 Nested Structs

Test cases with deeply nested structs time out during runtime due to having to write the header for every field at allocation time. For example, the test `brachiosaurus-struct-stress2` contains a struct with 2000 levels. Because we have to generate code to insert headers into every field, even nested ones, allocating a 2000-level struct 2000 times causes a timeout, as we end up with about 4,000,000 instructions just for inserting headers.

4.2.2 Stack Frame Size

We found that many test cases in `l3-large` fail due to large amounts of recursion. As discussed in Section 5.3.2, due to the typing annotations added to the stack frame, we are much more prone to stack overflows.

4.2.3 Array Compatibility

As described in section 2.1, the layout of arrays was modified to better align with the layout of other heap objects. As a result, arrays no longer match the C standard for arrays, and tests which link against C functions using C0 arrays now fail. One example of such a test is `helloworld.l4` from `l4-basic`:

```
//test return 11

int main() {
  int[] A = alloc_array(int, 3);
  A[0] = 0x6C6C6548; //  'l' 'l' 'e' 'H'
  A[1] = 0x6F57206F; //  'o' 'W' ' ' 'o'
  A[2] = 0x00646C72; // '\0' 'd' 'l' 'r'
  // puts(A); <- May be useful for testing, but interferes with autograder
  return strlen(A);
}
```

While C arrays are laid out such that the pointer stored in `A` would point to the first element of the array, with the length of the array 8 bytes behind it, our arrays are such that `A` now points to the length of the array, and the first element is 24 bytes ahead. For example, the test case `adalovelace-rotation_num_functions_2_num_args_150` from `l3-large` both allocates 150 temps and recurses about 150 times, which causes a stack overflow.

4.2.4 Stress Tests

Because writing to each temp requires several extra writes to update the stack frame, many stress tests now time out during runtime. For example, the test case `stegosaurus-return06` from `l2-basic` does many computations in a loop:

```
//test return 258226308
// Counts the number of non-negative lattice points within a circle of
   radius x around the origin.

int main () {

  int x = 25642;
  int i;
  int j;
  int result = 0;

  for (i = 0; i <= x; i++) {
    for (j = i; (i*i + j*j) <= x*x; j++) {
      result++;
    }
  }
  return result;
}
```

Now that each calculation done inside the loop guard and in the result needs to update the stack frame's pointer map, this test case times out.

5 Analysis

We will evaluate the collector using the goals for our collector defined in Section 4.1:

1. **Correctly execute code from L1 through L4.** As described in Section 4.2, our compiler now fails on several test cases from L2 through L4 due to a number of reasons. This is expected since adding garbage collection adds overhead to compile time, run time, and in our case stack usage. However, we might have been able to avoid the extra use of the stack by using a heuristic to identify pointers rather than storing the type of every temp (or as Runming described it, we should only collect things that "look like pointers").

2. **Collect objects when no pointers exist to them.** We were able to verify that the Cheney scan does collect objects correctly using the `heap`. If given a set of root pointers, our `collect` function does collect anything not reachable from that set.

3. **Avoid collecting objects that might be used again.** Several of our test cases were designed to force collections and test whether objects that might be used again were collected. Since our compiler passed these test cases, the collector appears not to collect any objects until their pointers are either overwritten by other data that was allocated to the same stack slot, or go out of scope by returning.

4. **Perform correctly under stress.** While our compiler has difficulty with stress tests from L2, L3, and L4 due to the extra overhead added from garbage collection, it is able to perform thousands of garbage collections within a single program.

5. **Avoid memory leaks.** Because of the typing annotations we store in the stack, we are able to identify all pointers in the program. This means that we cannot misidentify any other data as a pointer that might prevent us from collecting something in the heap.

5.1 Performance by Heap Limit

Uses (Bytes)	Frees Count	Frees (Bytes)	Heap size (Bytes)	Avg (ms)
816	0	0	900	36.2
64	1	752	800	34.6
160	1	656	700	37
256	1	560	600	39.8
352	1	464	500	41.2
112	2	704	400	37.1
48	3	768	300	37.1
176	4	640	200	38.7
48	12	768	100	39.6
48	48	768	50	44.3
err	err	err	25	err

Table 1: stack_iter_inline.l4 test case ran and timed with different heap size allocations

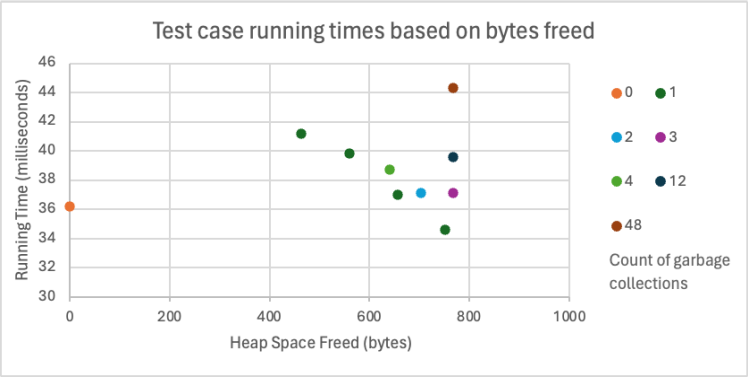

Figure 6: stack_iter_inline.l4 test case timing results when run with varying heap space availability

```
int main () {
  int *p = alloc(int);
  *p = 10;

  // allocate a bunch of space
  // force gc to remap
  for(int i=0;i<space_to_alloc;i++)
  {
    int* q=alloc(int);
  }

  assert(*p==10);

  return 0;
}
```

Uses (Bytes)	Garbage Collection Activation Count	Freed (Bytes)	Initial Heap size (Bytes)	Avg (ms)
1800	0	0	2000	33.7
504	1	1296	1700	38.4
504	1	1296	1500	37.6
936	1	864	1300	38.4
936	1	864	1100	38
504	3	1296	900	42.5
504	3	1296	700	39.8
err	err	err	500	err

Table 2: stack_iter_large.l4 test case ran and timed with different heap size allocations

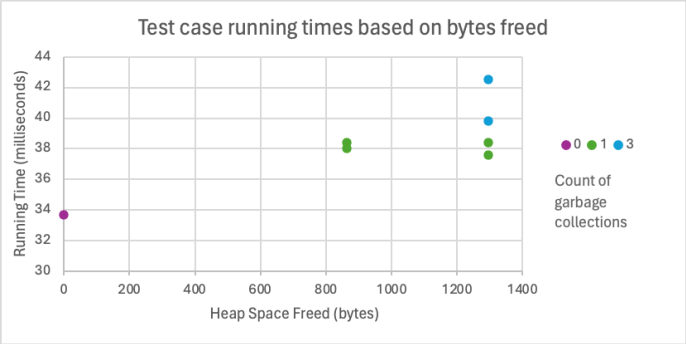

Figure 7: stack_iter_large.l4 test case timing results when run with varying heap space availability

```
// alloc ~400 bytes of dead space
void alloc_dead_space()
{
  int[] array=alloc_array(int, 100);
}

int main () {
  int size = 10;

  int[] arr=alloc_array(int, size);
   for(int i=0;i<size;i++)
   {
     arr[i]=5;
   }

  for(int i=0;i<space_to_alloc;i++)
  {
    alloc_dead_space();
  }

  int sum = 0;
  for(int i=0;i<size;i++)
  {
    sum+=arr[i];
  }

  assert(sum == size*5);

  return 0;
}
```

The benchmarking tests aimed to test both correctness and efficiency of the garbage collection algorithm. Generally, the tests followed the following pattern:

1. Initialize the allocatable heap space to some constant size
2. Initialize heap-allocated data to some value
3. Allocate a large amount of heap-space, forcing the garbage collector to remap the existing data
4. Assert the initially allocated data's correctness

With this test case design layout, we were able to check our garbage collector for both correctness and efficiency. By varying the allocatable heap space, we induced varying counts of garbage collection instances during the runtime of the sample test cases. This allowed us to track the performance of the garbage collector for both activation overhead and in relation to the actual number of bytes freed throughout the program.

To get this data, each program was ran and timed at least 10 times for each allocated heap size.

As can be seen from 7, we generally observed increases in runtime with increases in the counts of freed bytes. This is expected, as the garbage collector introduces multiple additional iterations over entire stack and heap allocated program space. Both the number of bytes freed and the overhead from initiating a garbage collection instance seemed to have a large influence over the program runtime than.
Interestingly, 6 seems to show a decreased runtime between test cases with like counts of garbage collection instances but larger counts of freed bytes. This is likely explained by the overhead iterations required to remap all non-garbage collected heap objects to new memory addresses with every iteration of garbage collection.

5.2 Graphing Utility

We additionally provide a utility for graphing the amount of data usage over time. In particular, rather than running with the `exe` flag, the amount of data used can be graphed using the `graph` flag like so:

```
bin/c0c --graph lab6-tests/TEST_NAME
./lab6-tests/TEST_NAME.exe
python3 graph.py
```

This will display the graph of memory used over time. **It is not recommended to run this on the stress tests. They produce extremely large files (over 1 GB).**

As an example, Figure 8 is the graph generated by running `stress03.14` with a 5000 byte memory limit. It is clear by the oscillation that every other call to `allocate_array` results in a collection.

5.3 Limitations and Potential Improvements

5.3.1 Conservative Collection

At the moment, objects on the heap are only collected once no stack variables point to them. While this ensures that no objects that are still in use are collected, this is overly conservative, since stack variables might not be used later in a function. Therefore, in the case of longer functions, garbage collection might be overly conservative and not collect objects that will never be used again.

In order to make our garbage collection more aggressive, we can use liveness information about our variables to collect objects where their pointers are either out of scope or no longer live. By running liveness analysis, we can identify where a temp is no longer live and mark it as no longer a pointer in the typing annotations. This would prevent it from appearing in the root pointer set, potentially allowing the object it points to to be garbage collected.

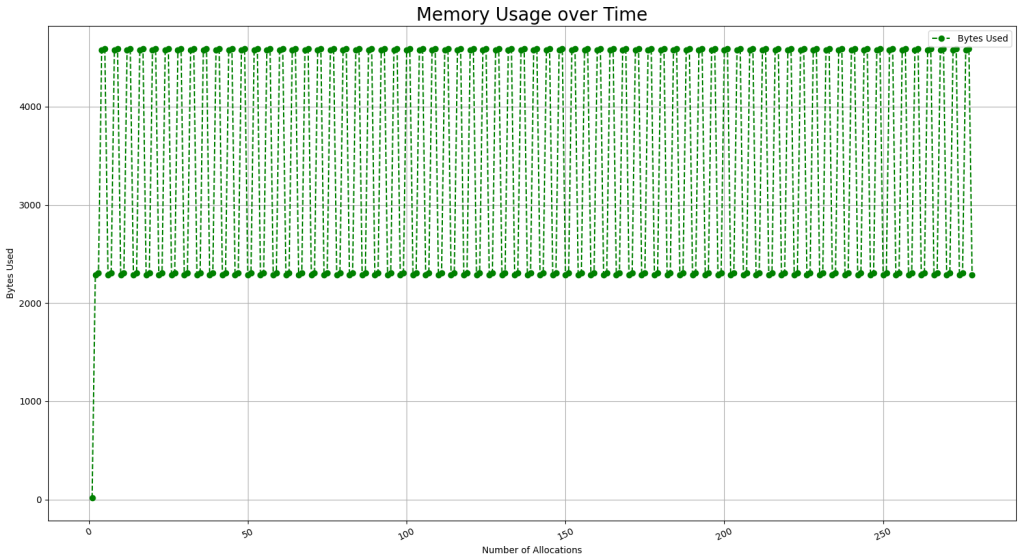

Figure 8: A graph of the number of allocations performed against number of bytes currently in use.

5.3.2 Stack Frame Size

Because we store on the stack whether or not each temp on the stack is a pointer, and additionally force all of our pointers to be stored on the stack, the size of each stack frame is much larger than it used to be. As a result, heavily recursive functions result in stack overflows, despite the addition of garbage collection.

Further, the maintenance of this typing information tripled the number of operations on pointer variables, since every pointer move required an update to the destination's typing information. Thus, we experienced slow down in test-case running times, leading to $\tilde{1}00$ more timeouts in our L6 compiler than in our L4 compiler.

To combat this, we have two potential improvements that we could implement:

1. We could halve the size of our typing annotations. Because the only relevant part of a temp's type is whether the temp is heap-allocated, we could store the status of each temp in only one bit. This would shrink the size of the typing annotations section on the stack, which could improve the amount of stack space being used.

2. Alternatively, we could stop storing temp information on the stack at all, instead storing information about our temps on the heap. One way we might do this is by keeping a linked list representing the function stack (and with relevant pointers), which we can traverse during our stack traversal function. While this will decrease the amount of stack space, this might result in slower runtime since we would be doing many stores into the heap, sacrificing locality.

3. To increase speed, the usage of registers throughout functions is able to be predicted at runtime within our compiler. Thus, to avoid having to force pointers onto the stack for the purpose of avoiding register data overwrites from the garbage collector, the protocol could be compiled or directly injected as part of the assembly of the program instead of being implemented in C. This would allow direct register data and typing information maintenance from the garbage collector, allowing more flexibility in the use of registers for pointers.

5.3.3 Heap Usage and Header Compression

Because of the 8-byte headers we added to every heap-allocated object, heap-allocated objects now take up much more space on the heap than they need to. Only two bits are used in the header to indicate the tag, and the rest of the header goes to waste. On arrays and structs, this is worse, since element headers and sizes need to be stored as well.

For arrays, one potential way to save space is by compressing the header so that more information is packed into the 8 bytes. Since element headers also only take 2 bits, we can store the tag in the same place as the array's header. This would require us to extract the last two bits of headers when checking them, but would cut down on 8 bytes of space.

For structs, because all sizes are 8-byte aligned, the last two bits of any size are guaranteed to be zero. This means that we can store field headers in the same 8 bytes as we store field sizes, which would significantly shrink the size of structs with many fields.

The proposed changes are shown in figures 9 and 10.

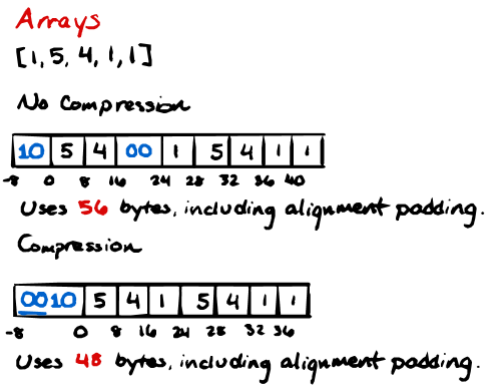

Figure 9: Array compression. Blue indicates binary numbers.

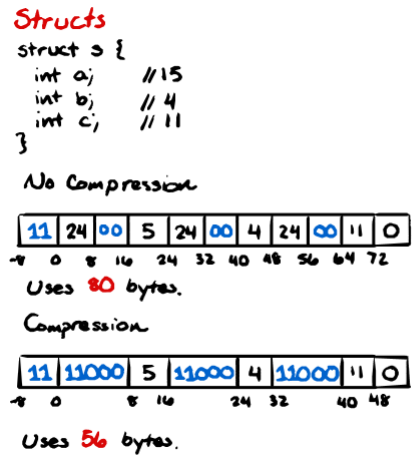

Figure 10: Array compression. Blue indicates binary numbers.

6 Appendix

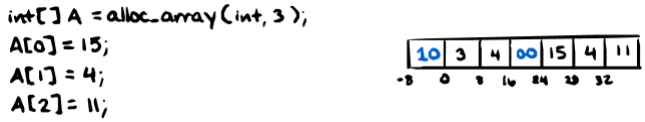

Figure 11: A simple `int` array.

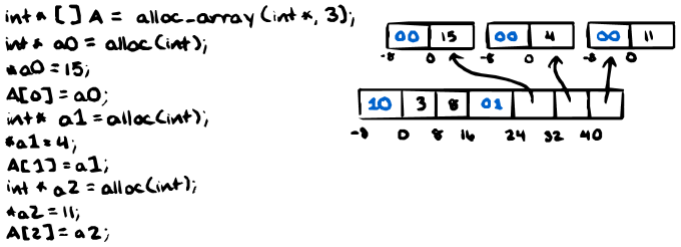

Figure 12: An `int*` array.

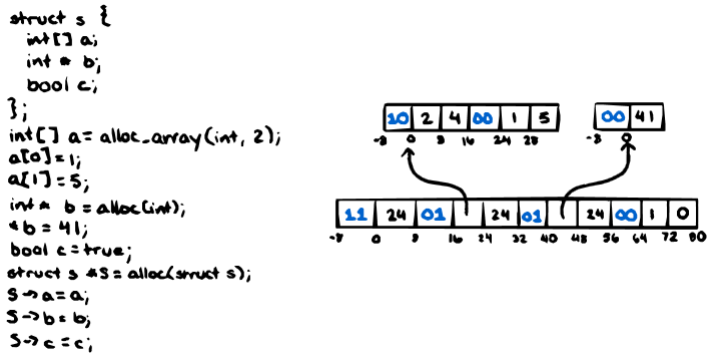

Figure 13: An array of simple structs.

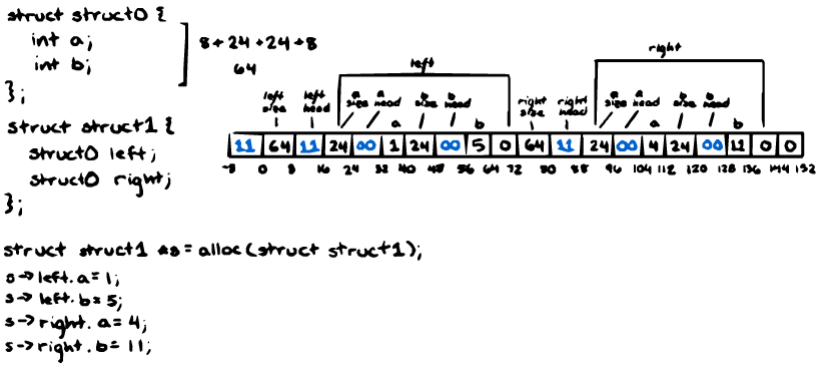

Figure 14: A nested struct.